Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

“Вдосконалення структури коду графічного редактора об’єктів”

Виконав:

студент групи ІМ-о51
Тараєв Олексій Богданович
Номер в списку - 14

Перевірив:

Рекечинський Д.О.

Київ 2026

Завдання:
Метою роботи є модернізація структури коду графічного редактора об'єктів для забезпечення масштабованості та зручного додавання нових типів графічних примітивів.

Індивідуальні вимоги за варіантом 14:

Керування пам'яттю (п. 1): Оскільки номер варіанта є парним ($Ж = 14$), клас контролера реалізується як динамічний об'єкт MyEditor із забезпеченням явного життєвого циклу — динамічним виділенням пам'яті/створенням екземпляра при запуску та коректним звільненням ресурсів (знищенням) при закритті вікна програми.

Графічні параметри (п. 3): Кольори та стилі фігур зберігаються з попередньої лабораторної роботи (прямокутник із білим заповненням, прозорий контурний еліпс, динамічний масив місткістю $N = 115$). «Гумовий» слід для всіх 6 фігур виконується пунктирною лінією.

Нові типи геометричних форм (п. 4): Реалізовано дві нові складні фігури з використанням множинного успадкування:

1. *«Лінія з кружечками» (LineCirclesShape):* успадковує LineShape та EllipseShape. Відмалювання здійснюється повторним використанням методів LineShape.show() (відрізок) та EllipseShape.show() (кружечки на кінцях).
2. *«Каркас куба» (CubeShape):* успадковує RectShape та LineShape. Відображення реалізовано через виклики RectShape.show() (передня й задня грані) та LineShape.show() (чотири з'єднувальні бічні ребра).

Користувацький інтерфейс (п. 5): Панель інструментів Toolbar (зі спливаючими підказками ToolTip) та меню «Об'єкти» розширено до 6 інструментів із повною синхронізацією активного стану в заголовку вікна.

КОД:

editor.py

```
import logging
import tkinter as tk
from abc import ABC, abstractmethod
from typing import Any


from shape import (
    CubeShape,
    EllipseShape,
    LineCirclesShape,
    LineShape,
    PointShape,
    RectShape,
)


__all__ = [
    "CubeEditor",
    "Editor",
    "EllipseEditor",
    "LineCirclesEditor",
    "LineEditor",
    "PointEditor",
    "RectEditor",
    "ShapeEditor",
]


logger = logging.getLogger("Lab4.Editor")
```

```python
class Editor(ABC):

    @abstractmethod

    def on_lb_down(self, event: tk.Event) -> None:

        pass


    @abstractmethod

    def on_lb_up(self, event: tk.Event) -> None:

        pass


    @abstractmethod

    def on_mouse_move(self, event: tk.Event) -> None:

        pass


    @abstractmethod

    def on_paint(self, canvas: tk.Canvas) -> None:

        pass




class ShapeEditor(Editor):

    def __init__(self, owner: Any) -> None:

        self._owner = owner

        self._x_start: int = 0

        self._y_start: int = 0

        self._x_end: int = 0

        self._y_end: int = 0

        self._is_drawing: bool = False


    def on_paint(self, canvas: tk.Canvas) -> None:

        pass
```

```
    def _clear_rubber_band(self) -> None:
        self._owner.canvas.delete("rubber_band")



class PointEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("PointEditor mouse down at (%d, %d).", event.x,
event.y)
        shape = PointShape()
        shape.set(event.x, event.y, event.x, event.y)
        self._owner.add_shape(shape)

    def on_lb_up(self, event: tk.Event) -> None:
        pass

    def on_mouse_move(self, event: tk.Event) -> None:
        pass



class LineEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("LineEditor start drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
        self._y_end = event.y
```

```
    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()
        self._owner.canvas.create_line(
            self._x_start,
            self._y_start,
            self._x_end,
            self._y_end,
            fill="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )


    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("LineEditor end drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = False
        self._clear_rubber_band()
        shape = LineShape()
        shape.set(self._x_start, self._y_start, event.x, event.y)
        self._owner.add_shape(shape)
```

```
class RectEditor(ShapeEditor):

    def on_lb_down(self, event: tk.Event) -> None:

        logger.debug("RectEditor center placed at (%d, %d).", event.x,
event.y)

        self._is_drawing = True

        self._x_start = event.x

        self._y_start = event.y

        self._x_end = event.x

        self._y_end = event.y


    def on_mouse_move(self, event: tk.Event) -> None:

        if not self._is_drawing:

            return

        self._x_end = event.x

        self._y_end = event.y

        self._clear_rubber_band()

        radius_x = abs(self._x_end - self._x_start)

        radius_y = abs(self._y_end - self._y_start)

        self._owner.canvas.create_rectangle(

            self._x_start - radius_x,

            self._y_start - radius_y,

            self._x_start + radius_x,

            self._y_start + radius_y,

            fill="",

            outline="black",

            dash=(4, 2),

            width=1,

            tags="rubber_band",

        )
```

```
    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("RectEditor corner released at (%d, %d).",
event.x, event.y)
        self._is_drawing = False
        self._clear_rubber_band()
        radius_x = abs(event.x - self._x_start)
        radius_y = abs(event.y - self._y_start)
        shape = RectShape()
        shape.set(
            self._x_start - radius_x,
            self._y_start - radius_y,
            self._x_start + radius_x,
            self._y_start + radius_y,
        )
        self._owner.add_shape(shape)



class EllipseEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("EllipseEditor start drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
        self._y_end = event.y
```

```
    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()
        x_min = min(self._x_start, self._x_end)
        y_min = min(self._y_start, self._y_end)
        x_max = max(self._x_start, self._x_end)
        y_max = max(self._y_start, self._y_end)
        self._owner.canvas.create_oval(
            x_min,
            y_min,
            x_max,
            y_max,
            fill="",
            outline="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )


    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("EllipseEditor end drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = False
        self._clear_rubber_band()
```

```
        shape = EllipseShape()
        shape.set(self._x_start, self._y_start, event.x, event.y)
        self._owner.add_shape(shape)



class LineCirclesEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("LineCirclesEditor start drag at (%d, %d).",
event.x, event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
        self._y_end = event.y

    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()

        self._owner.canvas.create_line(
            self._x_start,
            self._y_start,
            self._x_end,
            self._y_end,
            fill="black",
            dash=(4, 2),
```

```
            width=1,
            tags="rubber_band",
        )

        radius = 5
        self._owner.canvas.create_oval(
            self._x_start - radius,
            self._y_start - radius,
            self._x_start + radius,
            self._y_start + radius,
            outline="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )
        self._owner.canvas.create_oval(
            self._x_end - radius,
            self._y_end - radius,
            self._x_end + radius,
            self._y_end + radius,
            outline="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )

    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
```

```
        logger.debug("LineCirclesEditor end drag at (%d, %d).",
event.x, event.y)
        self._is_drawing = False
        self._clear_rubber_band()
        shape = LineCirclesShape()
        shape.set(self._x_start, self._y_start, event.x, event.y)
        self._owner.add_shape(shape)



class CubeEditor(ShapeEditor):
    def on_lb_down(self, event: tk.Event) -> None:
        logger.debug("CubeEditor start drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = True
        self._x_start = event.x
        self._y_start = event.y
        self._x_end = event.x
        self._y_end = event.y


    def on_mouse_move(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        self._x_end = event.x
        self._y_end = event.y
        self._clear_rubber_band()


        x_min = min(self._x_start, self._x_end)
        y_min = min(self._y_start, self._y_end)
        x_max = max(self._x_start, self._x_end)
        y_max = max(self._y_start, self._y_end)
```

```
        dx = max((x_max - x_min) // 3, 5)
        dy = max((y_max - y_min) // 3, 5)

        self._owner.canvas.create_rectangle(
            x_min,
            y_min,
            x_max,
            y_max,
            outline="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )

        self._owner.canvas.create_rectangle(
            x_min + dx,
            y_min - dy,
            x_max + dx,
            y_max - dy,
            outline="black",
            dash=(4, 2),
            width=1,
            tags="rubber_band",
        )

        for vx, vy in [
            (x_min, y_min),
            (x_max, y_min),
            (x_min, y_max),
```

```
            (x_max, y_max),
        ]:
            self._owner.canvas.create_line(
                vx,
                vy,
                vx + dx,
                vy - dy,
                fill="black",
                dash=(4, 2),
                width=1,
                tags="rubber_band",
            )


    def on_lb_up(self, event: tk.Event) -> None:
        if not self._is_drawing:
            return
        logger.debug("CubeEditor end drag at (%d, %d).", event.x,
event.y)
        self._is_drawing = False
        self._clear_rubber_band()
        shape = CubeShape()
        shape.set(self._x_start, self._y_start, event.x, event.y)
        self._owner.add_shape(shape)
```

shape.py

```
import tkinter as tk
from abc import ABC, abstractmethod


__all__ = [
```

```
    "CubeShape",
    "EllipseShape",
    "LineCirclesShape",
    "LineShape",
    "PointShape",
    "RectShape",
    "Shape",
]



class Shape(ABC):
    def __init__(self) -> None:
        self.xs1: int = 0
        self.ys1: int = 0
        self.xs2: int = 0
        self.ys2: int = 0

    def set(self, x1: int, y1: int, x2: int, y2: int) -> None:
        self.xs1 = x1
        self.ys1 = y1
        self.xs2 = x2
        self.ys2 = y2

    @abstractmethod
    def show(self, canvas: tk.Canvas) -> None:
        pass



class PointShape(Shape):
```

```
    def show(self, canvas: tk.Canvas) -> None:
        canvas.create_oval(
            self.xs1 - 1,
            self.ys1 - 1,
            self.xs1 + 1,
            self.ys1 + 1,
            fill="black",
            outline="black",
            tags="shape",
        )


class LineShape(Shape):
    def show(self, canvas: tk.Canvas) -> None:
        canvas.create_line(
            self.xs1,
            self.ys1,
            self.xs2,
            self.ys2,
            fill="black",
            width=1,
            tags="shape",
        )


class RectShape(Shape):
    def __init__(self, fill_color: str = "white") -> None:
        super().__init__()
        self.fill_color = fill_color
```

```
    def show(self, canvas: tk.Canvas) -> None:
        x_min = min(self.xs1, self.xs2)
        y_min = min(self.ys1, self.ys2)
        x_max = max(self.xs1, self.xs2)
        y_max = max(self.ys1, self.ys2)
        canvas.create_rectangle(
            x_min,
            y_min,
            x_max,
            y_max,
            fill=self.fill_color,
            outline="black",
            width=1,
            tags="shape",
        )



class EllipseShape(Shape):
    def __init__(self, fill_color: str = "") -> None:
        super().__init__()
        self.fill_color = fill_color


    def show(self, canvas: tk.Canvas) -> None:
        x_min = min(self.xs1, self.xs2)
        y_min = min(self.ys1, self.ys2)
        x_max = max(self.xs1, self.xs2)
        y_max = max(self.ys1, self.ys2)
        canvas.create_oval(
```

```
            x_min,
            y_min,
            x_max,
            y_max,
            fill=self.fill_color,
            outline="black",
            width=1,
            tags="shape",
        )



class LineCirclesShape(LineShape, EllipseShape):
    def __init__(self) -> None:
        super().__init__()
        self.fill_color = "white"


    def show(self, canvas: tk.Canvas) -> None:
        orig_x1, orig_y1, orig_x2, orig_y2 = self.xs1, self.ys1,
self.xs2, self.ys2


        LineShape.show(self, canvas)


        radius = 5
        EllipseShape.set(
            self,
            orig_x1 - radius,
            orig_y1 - radius,
            orig_x1 + radius,
            orig_y1 + radius,
```

```
        )
        EllipseShape.show(self, canvas)


        EllipseShape.set(
            self,
            orig_x2 - radius,
            orig_y2 - radius,
            orig_x2 + radius,
            orig_y2 + radius,
        )
        EllipseShape.show(self, canvas)


        self.set(orig_x1, orig_y1, orig_x2, orig_y2)



class CubeShape(RectShape, LineShape):
    def __init__(self) -> None:
        super().__init__()
        self.fill_color = ""


    def show(self, canvas: tk.Canvas) -> None:
        orig_x1, orig_y1, orig_x2, orig_y2 = self.xs1, self.ys1,
self.xs2, self.ys2


        x_min = min(orig_x1, orig_x2)
        y_min = min(orig_y1, orig_y2)
        x_max = max(orig_x1, orig_x2)
        y_max = max(orig_y1, orig_y2)
```

```
        dx = max((x_max - x_min) // 3, 5)
        dy = max((y_max - y_min) // 3, 5)

        RectShape.set(self, x_min + dx, y_min - dy, x_max + dx, y_max -
dy)
        RectShape.show(self, canvas)

        RectShape.set(self, x_min, y_min, x_max, y_max)
        RectShape.show(self, canvas)

        LineShape.set(self, x_min, y_min, x_min + dx, y_min - dy)
        LineShape.show(self, canvas)

        LineShape.set(self, x_max, y_min, x_max + dx, y_min - dy)
        LineShape.show(self, canvas)

        LineShape.set(self, x_min, y_max, x_min + dx, y_max - dy)
        LineShape.show(self, canvas)

        LineShape.set(self, x_max, y_max, x_max + dx, y_max - dy)
        LineShape.show(self, canvas)

        self.set(orig_x1, orig_y1, orig_x2, orig_y2)
```

my_editor.py

```
import logging
import tkinter as tk
from collections.abc import Callable
```

```
from editor import (
    CubeEditor,
    EllipseEditor,
    LineCirclesEditor,
    LineEditor,
    PointEditor,
    RectEditor,
    ShapeEditor,
)
from shape import Shape


__all__ = ["MyEditor", "ShapeObjectsEditor"]


logger = logging.getLogger("Lab4.MyEditor")




class MyEditor:
    MAX_CAPACITY = 115


    def __init__(
        self,
        canvas: tk.Canvas,
        on_mode_change: Callable[[str, str], None] | None = None,
    ) -> None:
        logger.info("Initializing MyEditor instance (dynamic
lifecycle).")
        self.canvas = canvas
        self._on_mode_change = on_mode_change
        self._pcshape: list[Shape] = []
```

```
        self._editors: dict[str, ShapeEditor] = {
            "point": PointEditor(self),
            "line": LineEditor(self),
            "rect": RectEditor(self),
            "ellipse": EllipseEditor(self),
            "line_circles": LineCirclesEditor(self),
            "cube": CubeEditor(self),
        }
        self._shape_labels: dict[str, str] = {
            "point": "Крапка",
            "line": "Лінія",
            "rect": "Прямокутник",
            "ellipse": "Еліпс",
            "line_circles": "Лінія з кружечками",
            "cube": "Каркас куба",
        }

        self._current_type_name: str = "point"
        self._current_editor: ShapeEditor =
self._editors[self._current_type_name]

    @property
    def current_type_name(self) -> str:
        return self._current_type_name

    @property
    def current_label(self) -> str:
        return self._shape_labels.get(self._current_type_name, "")
```

```
    def start_editor(self, shape_type: str) -> None:
        if shape_type in self._editors:
            logger.info("Switching mode to %s.", shape_type)
            self._current_editor = self._editors[shape_type]
            self._current_type_name = shape_type
            if self._on_mode_change is not None:
                self._on_mode_change(shape_type, self.current_label)
        else:
            logger.error("Unknown editor type requested: '%s'.",
shape_type)

    def on_lb_down(self, event: tk.Event) -> None:
        self._current_editor.on_lb_down(event)

    def on_lb_up(self, event: tk.Event) -> None:
        self._current_editor.on_lb_up(event)

    def on_mouse_move(self, event: tk.Event) -> None:
        self._current_editor.on_mouse_move(event)

    def on_paint(self) -> None:
        self.canvas.delete("shape")
        for shape in self._pcshape:
            shape.show(self.canvas)

    def add_shape(self, shape: Shape) -> None:
        if len(self._pcshape) < self.MAX_CAPACITY:
            self._pcshape.append(shape)
```

```
            logger.info(
                "Shape added to dynamic array. Current size: %d/%d.",
                len(self._pcshape),
                self.MAX_CAPACITY,
            )
            self.on_paint()
        else:
            logger.warning("Dynamic array capacity reached (%d).", self.MAX_CAPACITY)


    def on_init_menu_popup(self, menu_var: tk.StringVar) -> None:
        menu_var.set(self._current_type_name)
        logger.debug("Menu state synced: current mode '%s'.", self._current_type_name)


    def destroy(self) -> None:
        logger.info("Destroying MyEditor: cleaning shapes and references.")
        self.canvas.delete("all")
        self._pcshape.clear()
        self._editors.clear()



ShapeObjectsEditor = MyEditor
```

toolbar.py

```
import logging
import math
import tkinter as tk
from collections.abc import Callable
```

```
from tkinter import ttk


__all__ = ["ToolTip", "Toolbar"]


logger = logging.getLogger("Lab4.Toolbar")




class ToolTip:
    def __init__(
        self,
        widget: tk.Widget,
        text: str,
        notify_callback: Callable[[str], None] | None = None,
    ) -> None:
        self._widget = widget
        self._text = text
        self._notify_callback = notify_callback
        self._tip_window: tk.Toplevel | None = None


        self._widget.bind("<Enter>", self.on_notify_enter)
        self._widget.bind("<Leave>", self.on_notify_leave)


    def on_notify_enter(self, _event: tk.Event) -> None:
        if self._notify_callback is not None:
            self._notify_callback(self._text)
        if self._tip_window is not None:
            return
        x = self._widget.winfo_rootx() + 10
        y = self._widget.winfo_rooty() + self._widget.winfo_height() +
```

```
4
        self._tip_window = tk.Toplevel(self._widget)
        self._tip_window.wm_overrideredirect(True)
        self._tip_window.wm_geometry(f"+{x}+{y}")
        label = tk.Label(
            self._tip_window,
            text=self._text,
            background="#FFFFE0",
            foreground="#000000",
            relief=tk.SOLID,
            borderwidth=1,
            font=("Segoe UI", 9),
            padx=6,
            pady=2,
        )
        label.pack()

    def on_notify_leave(self, _event: tk.Event) -> None:
        if self._tip_window is not None:
            self._tip_window.destroy()
            self._tip_window = None



class Toolbar(ttk.Frame):
    def __init__(
        self,
        parent: tk.Widget,
        on_select_tool: Callable[[str], None],
    ) -> None:
```

```
        super().__init__(parent, relief=tk.RAISED, borderwidth=1)
        self._on_select_tool = on_select_tool
        self._icons: dict[str, tk.PhotoImage] = {}
        self._buttons: dict[str, tk.Button] = {}
        self._tooltips: list[ToolTip] = []

        self._build_icons()
        self._build_buttons()

    def _build_icons(self) -> None:
        size = 24

        point_img = tk.PhotoImage(width=size, height=size)
        for y in range(size):
            for x in range(size):
                if (x - 12) ** 2 + (y - 12) ** 2 <= 9:
                    point_img.put("#000000", (x, y))
        self._icons["point"] = point_img

        line_img = tk.PhotoImage(width=size, height=size)
        for i in range(4, 20):
            line_img.put("#000000", (i, 23 - i))
            line_img.put("#000000", (i, 22 - i))
        self._icons["line"] = line_img

        rect_img = tk.PhotoImage(width=size, height=size)
        for y in range(5, 19):
            for x in range(4, 20):
                if x in (4, 19) or y in (5, 18):
```

```
                rect_img.put("#000000", (x, y))
            else:
                rect_img.put("#FFFFFF", (x, y))
        self._icons["rect"] = rect_img

        ellipse_img = tk.PhotoImage(width=size, height=size)
        rx, ry = 8.0, 5.5
        cx, cy = 12.0, 12.0
        for angle in range(360):
            rad = math.radians(angle)
            x = round(cx + rx * math.cos(rad))
            y = round(cy + ry * math.sin(rad))
            ellipse_img.put("#000000", (x, y))
        self._icons["ellipse"] = ellipse_img

        lc_img = tk.PhotoImage(width=size, height=size)
        for i in range(7, 17):
            lc_img.put("#000000", (i, 23 - i))
        for dy in range(-2, 3):
            for dx in range(-2, 3):
                if dx * dx + dy * dy <= 4:
                    lc_img.put("#000000", (6 + dx, 17 + dy))
                    lc_img.put("#000000", (17 + dx, 6 + dy))
        self._icons["line_circles"] = lc_img

        cube_img = tk.PhotoImage(width=size, height=size)

        for x in range(4, 16):
            cube_img.put("#000000", (x, 10))
```

```
            cube_img.put("#000000", (x, 20))
        for y in range(10, 21):
            cube_img.put("#000000", (4, y))
            cube_img.put("#000000", (15, y))

        for x in range(8, 20):
            cube_img.put("#000000", (x, 5))
            cube_img.put("#000000", (x, 15))
        for y in range(5, 16):
            cube_img.put("#000000", (8, y))
            cube_img.put("#000000", (19, y))
        for step in range(5):
            cube_img.put("#000000", (4 + step, 10 - step))
            cube_img.put("#000000", (15 + step, 10 - step))
            cube_img.put("#000000", (4 + step, 20 - step))
            cube_img.put("#000000", (15 + step, 20 - step))
        self._icons["cube"] = cube_img

    def _build_buttons(self) -> None:
        tools = [
            ("point", "Крапка"),
            ("line", "Лінія"),
            ("rect", "Прямокутник"),
            ("ellipse", "Еліпс"),
            ("line_circles", "Лінія з кружечками"),
            ("cube", "Каркас куба"),
        ]
        for shape_type, tooltip_text in tools:
            btn = tk.Button(
```

```
                self,
                image=self._icons[shape_type],
                width=28,
                height=28,
                relief=tk.RAISED,
                bd=1,
                command=lambda t=shape_type:
self._on_button_pressed(t),
            )
            btn.pack(side=tk.LEFT, padx=2, pady=2)
            self._buttons[shape_type] = btn
            self._tooltips.append(ToolTip(btn, tooltip_text,
self.on_notify))

    def _on_button_pressed(self, shape_type: str) -> None:
        logger.info("Toolbar button clicked: '%s'.", shape_type)
        self._on_select_tool(shape_type)

    def on_notify(self, tooltip_text: str) -> None:
        logger.debug("OnNotify triggered: displaying tooltip '%s'.",
tooltip_text)

    def set_active_button(self, shape_type: str) -> None:
        for key, btn in self._buttons.items():
            if key == shape_type:
                btn.config(relief=tk.SUNKEN, bg="#D0E0F0")
            else:
                btn.config(relief=tk.RAISED, bg="SystemButtonFace")
```

main.py

```
import logging
import tkinter as tk
from tkinter import messagebox


from my_editor import MyEditor
from toolbar import Toolbar


logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s [%(levelname)s] (%(name)s): %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger("Lab4.Main")



class MainWindow(tk.Tk):
    BASE_TITLE = "OOP_lab4"


    def __init__(self) -> None:
        super().__init__()
        logger.info("Initializing MainWindow.")
        self.title(self.BASE_TITLE)
        self.geometry("780x560")


        self._menu_shape_var = tk.StringVar(value="point")


        self._init_toolbar()
        self._init_canvas()
```

```
        self._editor: MyEditor | None = None
        self._create_dynamic_editor()

        self._init_menu()
        self._bind_events()

        self.protocol("WM_DELETE_WINDOW", self._on_close)

        if self._editor is not None:
            self._editor.start_editor("point")

    def _create_dynamic_editor(self) -> None:
        logger.info("Dynamically allocating MyEditor instance.")
        self._editor = MyEditor(
            self._canvas,
            on_mode_change=self._on_editor_mode_changed,
        )

    def _destroy_dynamic_editor(self) -> None:
        if self._editor is not None:
            logger.info("Dynamically deallocating/destroying MyEditor
instance.")
            self._editor.destroy()
            del self._editor
            self._editor = None

    def _init_toolbar(self) -> None:
        logger.debug("Configuring Toolbar.")
```

```
        self._toolbar = Toolbar(
            self,
            on_select_tool=lambda shape_type: (
                self._editor.start_editor(shape_type)
                if self._editor is not None
                else None
            ),
        )
        self._toolbar.pack(side=tk.TOP, fill=tk.X)

    def _init_canvas(self) -> None:
        self._canvas = tk.Canvas(self, bg="#FFFFFF",
cursor="crosshair")
        self._canvas.pack(side=tk.TOP, fill=tk.BOTH, expand=True)

    def _init_menu(self) -> None:
        logger.debug("Configuring menu bar.")
        menubar = tk.Menu(self)

        file_menu = tk.Menu(menubar, tearoff=0)
        file_menu.add_command(label="Вихід", command=self._on_close)
        menubar.add_cascade(label="Файл", menu=file_menu)

        objects_menu = tk.Menu(
            menubar,
            tearoff=0,
            postcommand=lambda: (
                self._editor.on_init_menu_popup(self._menu_shape_var)
                if self._editor is not None
```

```
                else None
            ),
        )
        tools = [
            ("point", "Крапка"),
            ("line", "Лінія"),
            ("rect", "Прямокутник"),
            ("ellipse", "Еліпс"),
            ("line_circles", "Лінія з кружечками"),
            ("cube", "Каркас куба"),
        ]
        for shape_type, label in tools:
            objects_menu.add_radiobutton(
                label=label,
                variable=self._menu_shape_var,
                value=shape_type,
                command=lambda t=shape_type: (
                    self._editor.start_editor(t) if self._editor is not
None else None
                ),
            )
        menubar.add_cascade(label="Об'єкти", menu=objects_menu)

        help_menu = tk.Menu(menubar, tearoff=0)
        help_menu.add_command(label="Про програму",
command=self._on_about)
        menubar.add_cascade(label="Довідка", menu=help_menu)

        self.config(menu=menubar)
```

```
    def _on_editor_mode_changed(self, shape_type: str, label_ukr: str)
-> None:
        self.title(f"{self.BASE_TITLE} - Об'єкт: {label_ukr}")
        self._menu_shape_var.set(shape_type)
        self._toolbar.set_active_button(shape_type)

    def _bind_events(self) -> None:
        self._canvas.bind(
            "<Button-1>",
            lambda e: self._editor.on_lb_down(e) if self._editor else
None,
        )
        self._canvas.bind(
            "<B1-Motion>",
            lambda e: self._editor.on_mouse_move(e) if self._editor
else None,
        )
        self._canvas.bind(
            "<ButtonRelease-1>",
            lambda e: self._editor.on_lb_up(e) if self._editor else
None,
        )
        self._canvas.bind(
            "<Configure>",
            lambda _e: self._editor.on_paint() if self._editor else
None,
        )

    def _on_about(self) -> None:
        logger.info("Displaying About dialog.")
        messagebox.showinfo(
```

```
            "Про програму",
            "Лабораторна робота №4\nВдосконалення структури коду графічного редактора об'єктів\nВаріант 14 (Ж = 14, динамічний MyEditor)",
        )

    def _on_close(self) -> None:
        logger.info("Closing application window.")
        self._destroy_dynamic_editor()
        self.destroy()


def main() -> None:
    logger.info("Starting application loop.")
    app = MainWindow()
    app.mainloop()
    logger.info("Application loop ended.")


if __name__ == "__main__":
    main()
```

Скріншоти:

Рис. 1. Головне вікно програми з оновленим інтерфейсом на 6 фігур

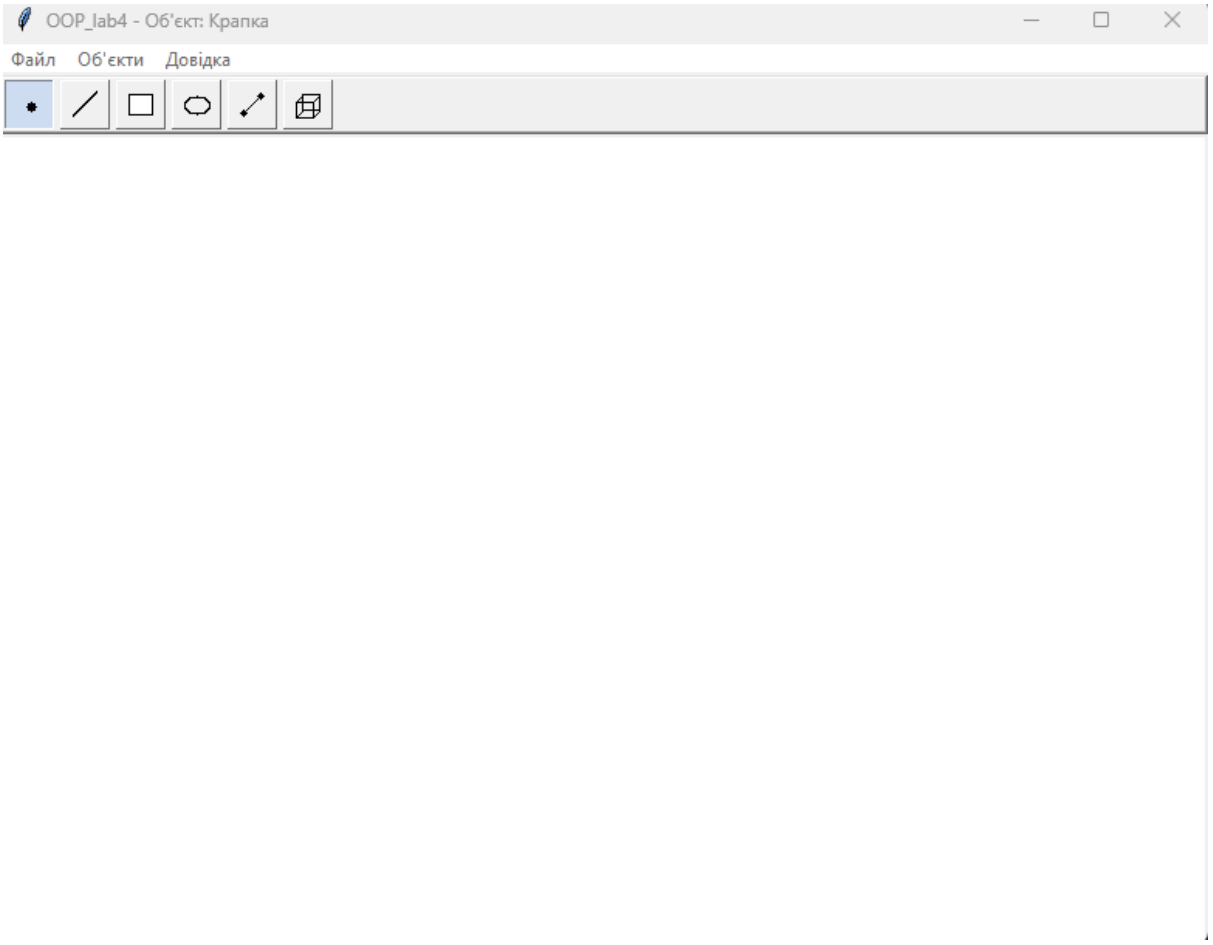


Рис. 2. Спливаюча підказка (ToolTip) нової фігури

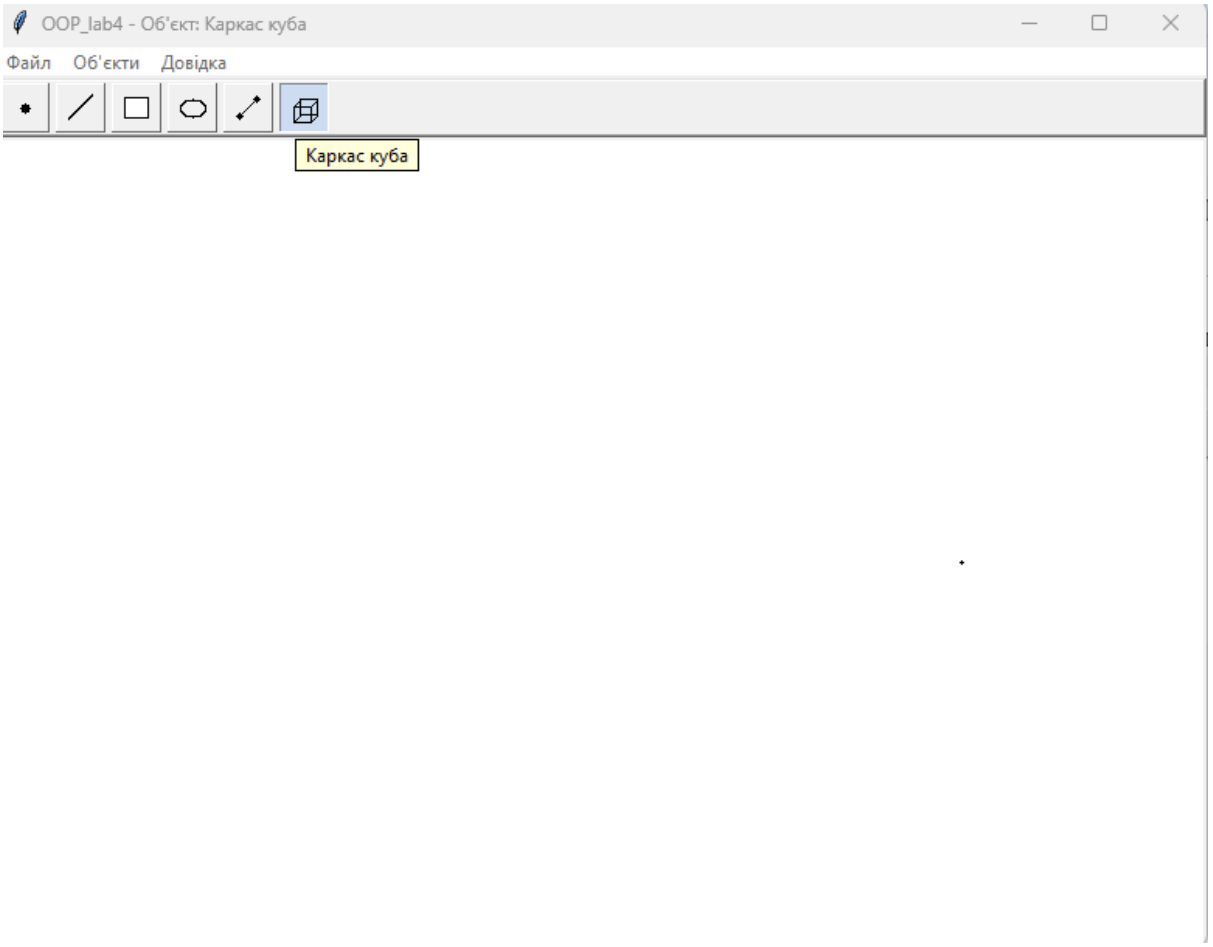


Рис. 3. Процес введення «Лінії з кружечками» з пунктирним слідом

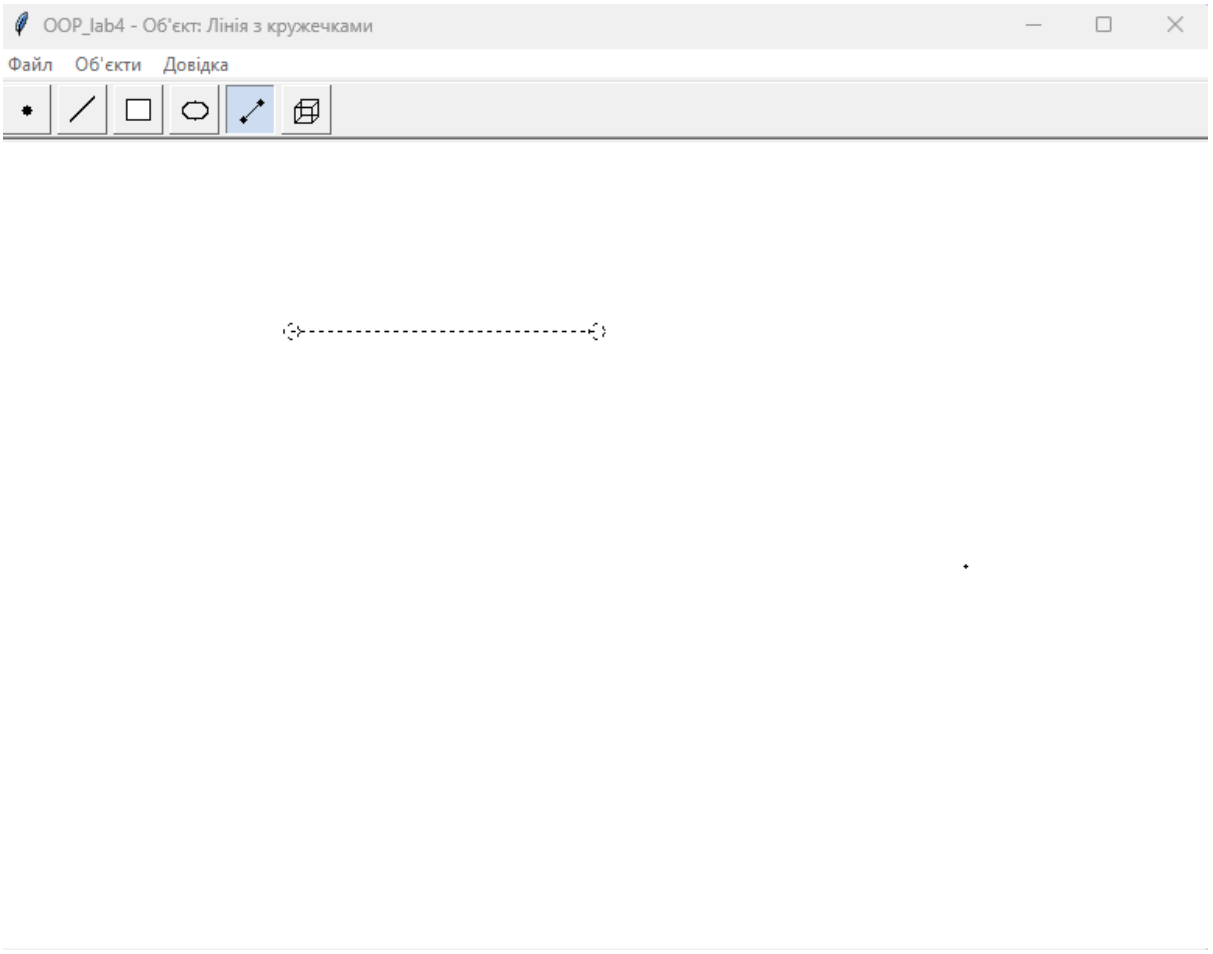


Рис. 4. Процес введення «Каркаса куба» з пунктирним слідом

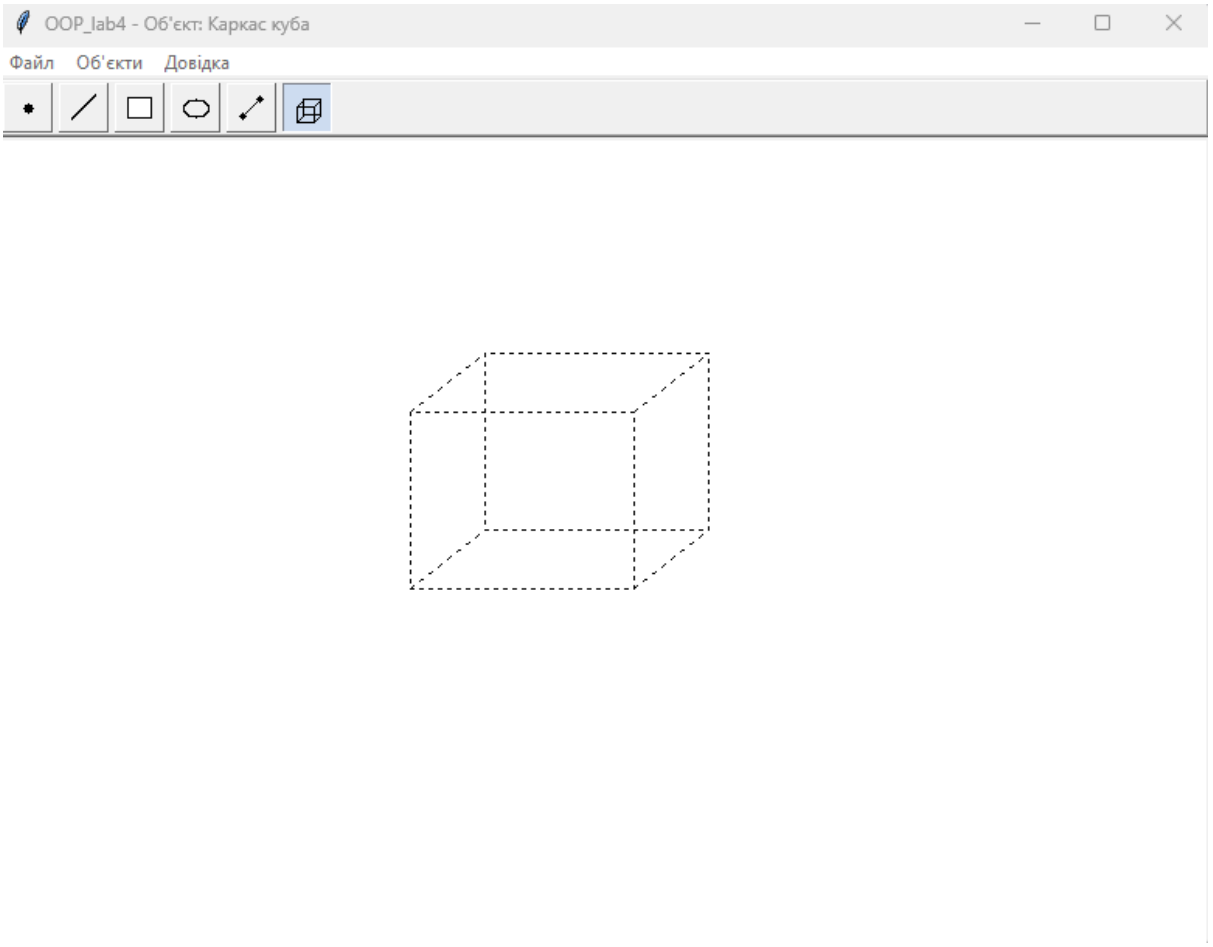


Рис. 5. Результуюче полотно з усіма шістьма типами фігур

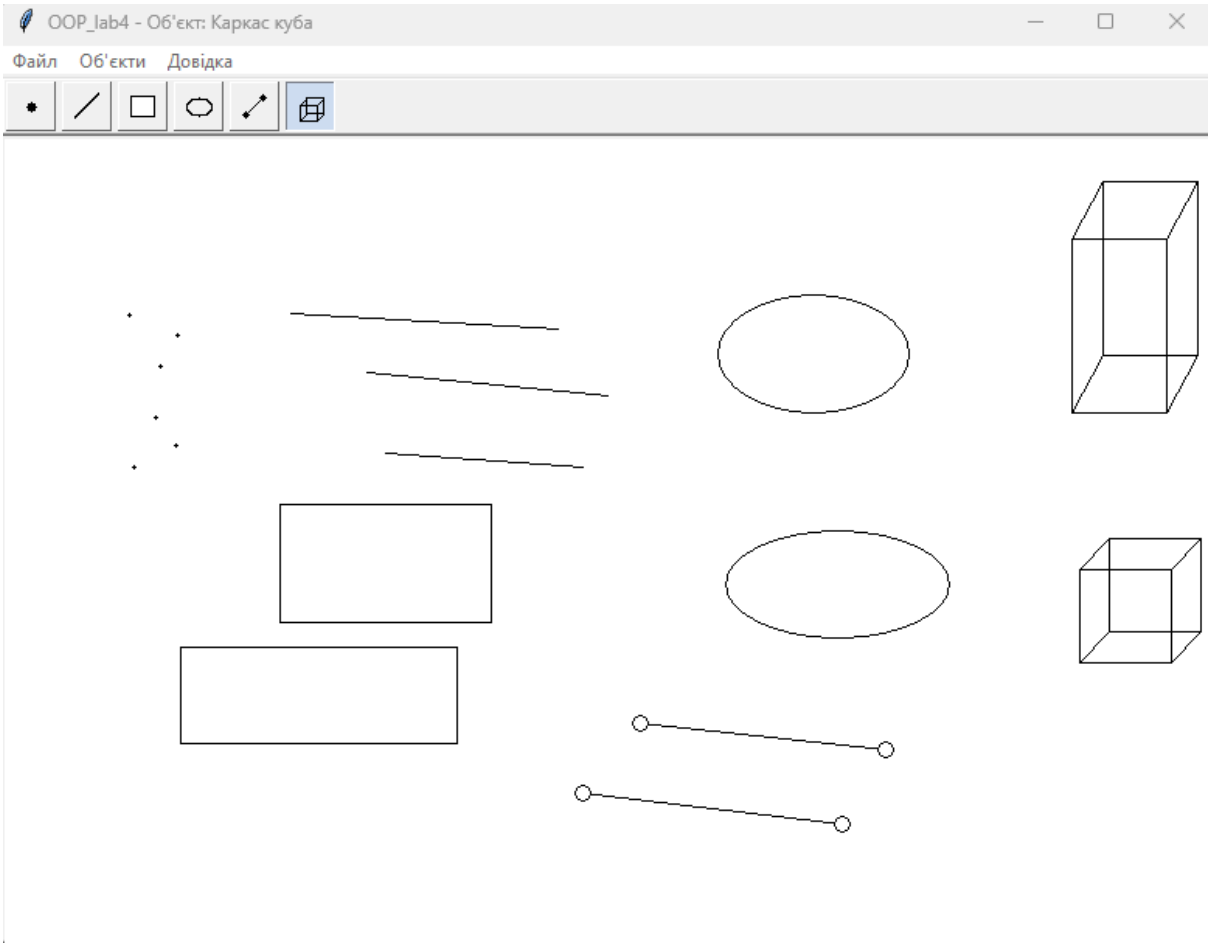
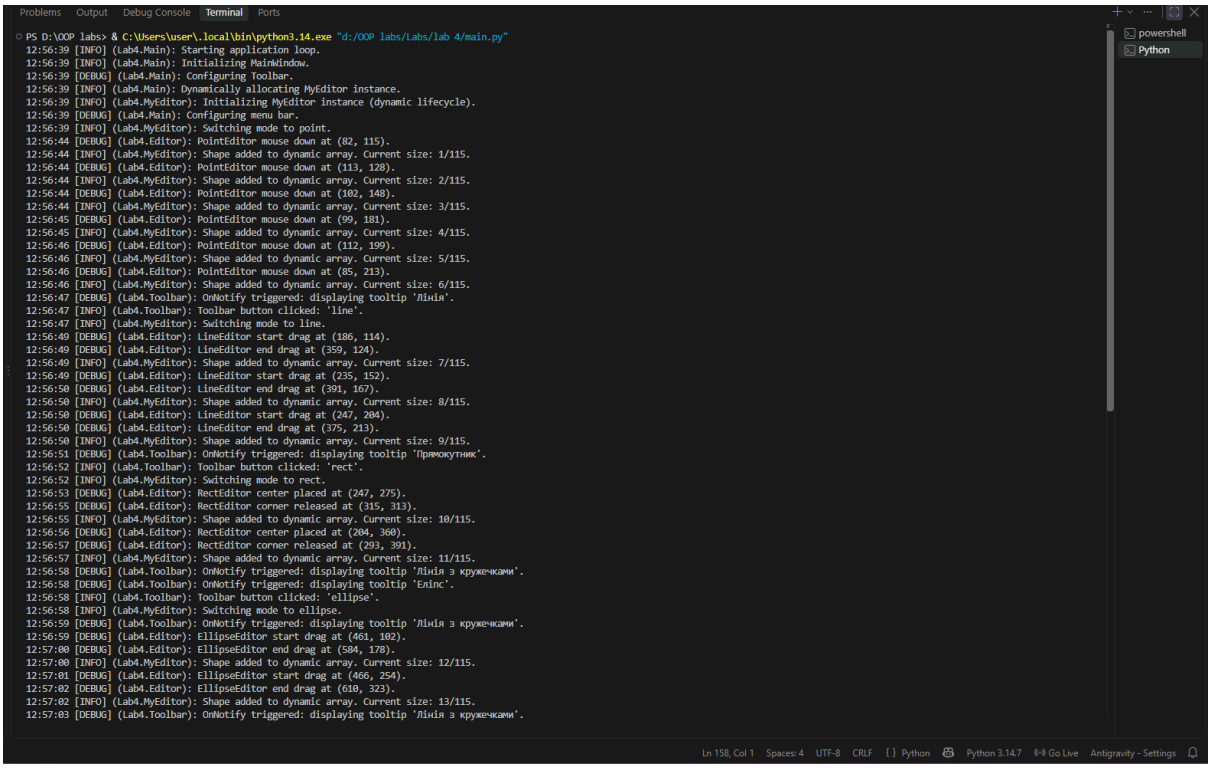
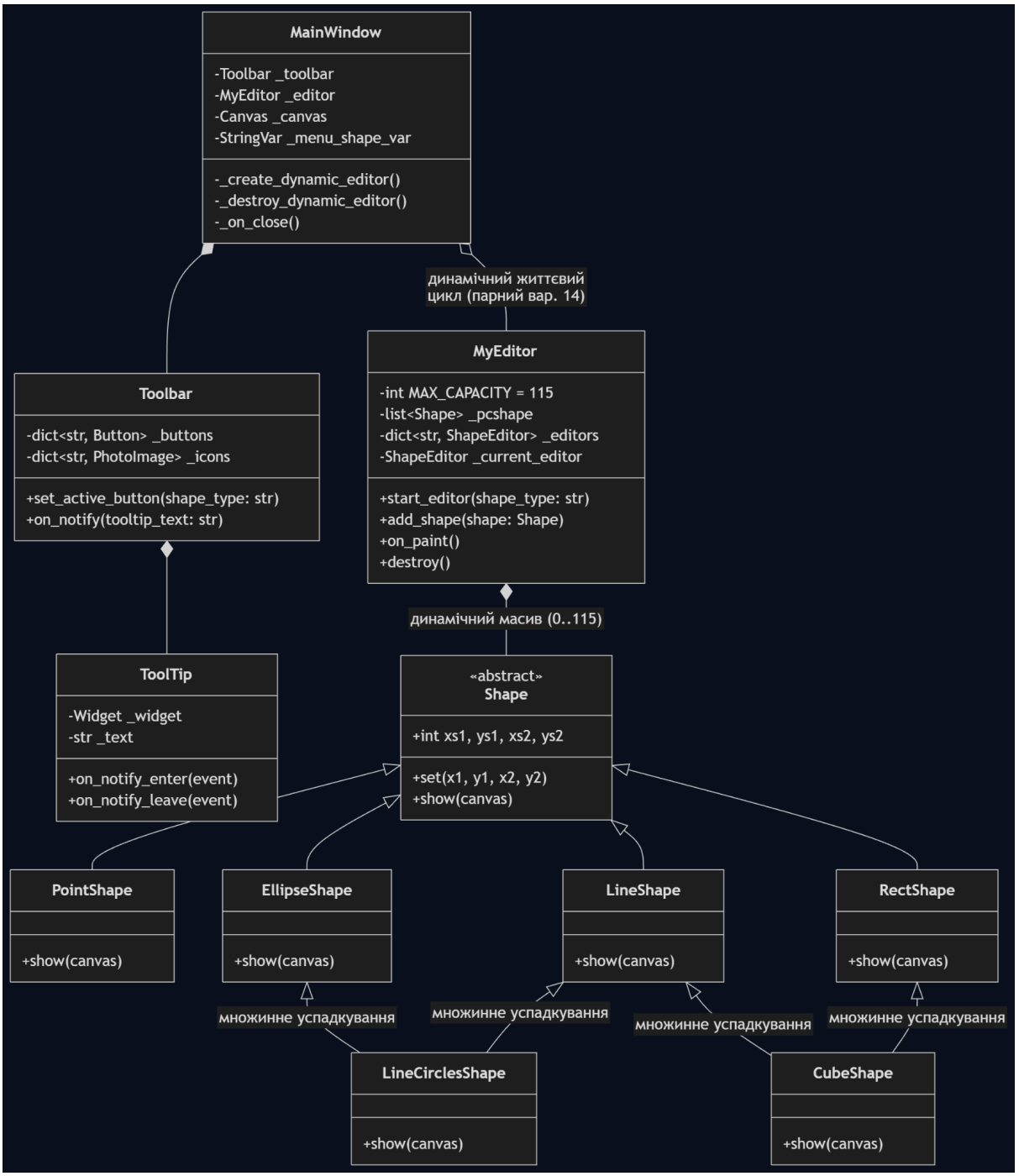


Рис. 6. Журнал консолі з підтвердженням життєвого циклу `MyEditor`



Діаграма класів та залежностей



Опис діаграми та взаємодії класів:
Клас MainWindow відповідає за створення та контрольоване знищення динамічного екземпляра MyEditor через методи _create_dynamic_editor() та _destroy_dynamic_editor().

Базова ієрархія примітивів Shape масштабована без дублювання базових механізмів: класи LineCirclesShape та CubeShape реалізують множинне успадкування і формують складні зображення, делегуючи промальовування методів show() своїм предкам.

MyEditor акумулює всі 6 редакторів у словнику _editors, забезпечуючи зв'язування між розширеною панеллю Toolbar (6 кнопок), випадаючим меню та полотном малювання.

Висновки

У ході лабораторної роботи №4 було вдосконалено структуру коду векторного графічного редактора об'єктів мовою Python.

Відповідно до парного варіанта ($Ж = 14$), контролер редактора реалізовано у вигляді динамічного об'єкта класу MyEditor із керованим життєвим циклом, що унеможливлює витоки ресурсів при закритті програми.

За допомогою множинного успадкування створено класи LineCirclesShape (на базі LineShape та EllipseShape) і CubeShape (на базі RectShape та LineShape), у яких графічне відображення досягнуто шляхом виклику методів show() відповідних базових класів без дублювання коду малювання. Архітектура проєкту підтвердила високу гнучкість: розширення функціоналу до 6 інструментів було реалізовано без переписування основних модулів системи.